ATHLETEOS

Personalized Gym Athlete Web App

TECHNICAL REQUIREMENTS DOCUMENT (TRD)

TECHNICAL THESIS Build a privacy-conscious, mobile-first fitness platform whose core tracking system is deterministic and auditable, while CV and personalization modules provide versioned, confidence-scored assistance rather than opaque control.

Document field	Value
Working product name	AthleteOS
Document	Technical Requirements Document
Version	1.0
Companion document	AthleteOS Product Requirements Document v1.0
Primary platform	Responsive web + installable PWA
Architecture baseline	Next.js + FastAPI + PostgreSQL + Redis + object storage
AI/CV baseline	MediaPipe Pose Landmarker; optional server-side Python inference
Date	11 August 2026

Bodybuilding • Calisthenics • Athletic Performance • Aesthetic Physique • Hybrid

Engineering specification for frontend, backend, data, AI/CV, infrastructure, security and quality.

00 Document Map

This TRD converts the AthleteOS PRD into an implementable technical specification. Requirement IDs are designed to be traceable into tickets, tests and release gates.

Architecture stance Start as a modular monolith with explicit domain boundaries. Split a domain into a separate service only when independent scaling, fault isolation, ownership or release cadence provides measurable value.

01 Technical Scope & Engineering Principles

1.1 In scope

- Authentication, athlete profile, onboarding, consent and settings.
- Timestamped body metrics, hydration, sleep/recovery and progress-photo metadata.
- Exercise catalogue, generated training plans, day/session execution, per-set logging, personal records and progression.
- Global + Indian food catalogue, aliases, recipes, meal logging, nutrition summaries and water logging.
- Excel-style recurring habit grid, completion logs and streak calculations.
- Weekly/monthly analytics, comparison reports and recommendation explanations.
- Optional CV pipeline for pose, segmentation, proportion and symmetry signals.
- Admin ingestion, content moderation, data quality and operational tools.
- Infrastructure, security, backups, observability, CI/CD and testing.

1.2 Engineering principles

ID	Principle	Technical implication
ENG-01	System of record first	PostgreSQL owns durable athlete state; cache and analytics views are disposable/rebuildable.
ENG-02	Explainable personalization	Every automated plan change stores rule/model version, evidence and human-readable reason.
ENG-03	Privacy by default	Photo processing is on-device when feasible; raw-image retention is opt-in and separately revocable.
ENG-04	No fake precision	Derived values carry confidence/source metadata and never overwrite manually measured facts.
ENG-05	Idempotent writes	Session completion, food import, report jobs and webhooks use idempotency keys.
ENG-06	Progressive complexity	Beginner UI hides advanced set attributes; schema still supports RIR/RPE, tempo, assistance and holds.
ENG-07	Version everything important	Plans, exercise definitions, nutrition sources, recommendation rules and CV features are versioned.
ENG-08	Mobile-first latency	Workout logging and habit ticking must remain usable on weak mobile networks; optimistic UI is allowed with durable retries.

1.3 Out of scope / guardrails

- No medical diagnosis, injury diagnosis or clinical body-composition claim.
- No face recognition or identity inference from athlete photos.
- No requirement for nude imagery. Capture guidance uses normal sportswear/fitted clothing; sensitive raw media is never mandatory.
- No LLM may directly mutate a training prescription without deterministic validation and an auditable recommendation record.
- No public social network is required for v1.

02 Target Architecture

The recommended v1 topology separates the web experience, API/domain logic, durable data, optional media, background jobs and AI/CV execution. The application can be deployed from one backend codebase while keeping boundaries explicit.

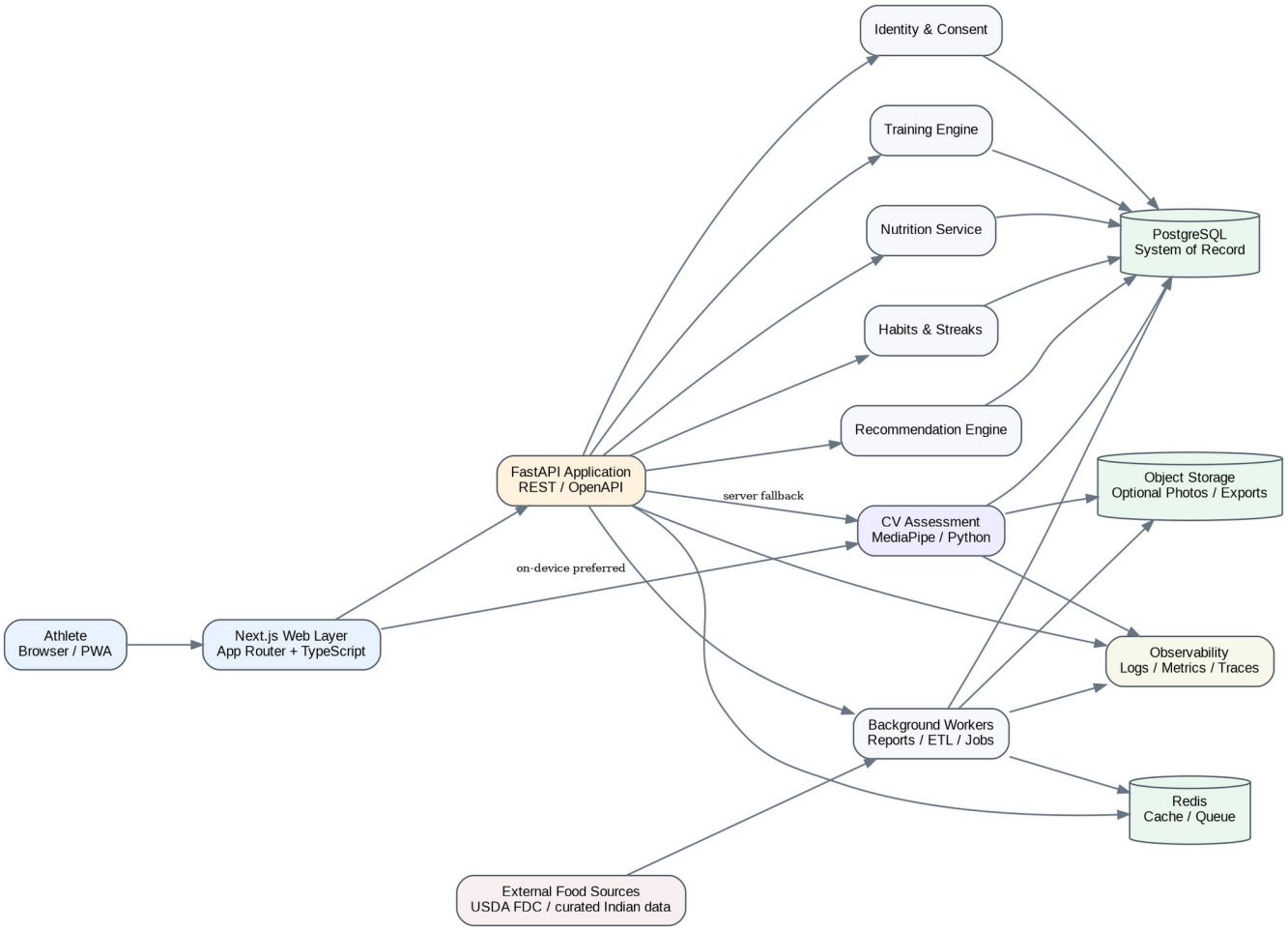

Figure 1. Target architecture - modular monolith with isolated AI/CV and background execution paths.

2.1 Logical components

Component	Responsibility	Scale characteristic
Web/PWA	Navigation, offline-friendly logging, charts, on-device CV, auth UI	Horizontal via CDN/edge
API application	Validation, authorization, orchestration and domain commands	Horizontal stateless replicas
Training module	Templates, plan generation, workout execution, progression	CPU/light DB
Nutrition module	Food search, recipes, meal logs, targets, summaries	Read/search heavy
Habit module	Schedules, completion matrix, streaks	Small transactional writes
Recommendation module	Evidence aggregation, rules/scoring, explanations	Batch + planning-boundary jobs
CV module	Quality gate, landmarks/segmentation, feature extraction	Device CPU/GPU; server fallback
Workers	ETL, imports, report generation, notifications, cleanup	Queue-driven independent scale

Component	Responsibility	Scale characteristic
PostgreSQL	System-of-record relational data	Primary + read replicas later
Redis	Cache, rate limit, locks and background queue broker	Memory-bound
Object storage	Optional progress media, exports, model assets	Capacity/egress bound

2.2 Request patterns

- Synchronous: profile reads/writes, workout logging, food search, habit tick, dashboard summary.
- Asynchronous: report generation, food-source import/normalization, photo deletion, notification fan-out, analytics aggregation.
- On-device: preferred CV inference and image preprocessing; upload only derived features when raw-image storage is disabled.
- Planning-boundary: training adaptations should generally be applied between sessions/weeks, not randomly mid-session.

03 Technology Stack & Service Boundaries

Layer	Recommended baseline	Rationale / notes
Frontend	Next.js App Router + TypeScript	Server/client composition, routing, PWA shell, strong ecosystem.
UI	Tailwind CSS + accessible component primitives	Fast responsive design; theme tokens; avoid coupling domain logic to UI kit.
Client data	TanStack Query or equivalent	Cache/retry/optimistic updates for set logs and habits.
API	FastAPI + Pydantic	Typed Python API, OpenAPI contract, natural fit with data/CV modules.
Persistence	PostgreSQL	Relational integrity, JSONB where justified, analytical SQL, indexes.
ORM/migrations	SQLAlchemy + Alembic	Explicit transactions and schema migrations.
Cache/jobs	Redis + Celery/RQ-class worker	Caching, distributed locks, queues and scheduled jobs.
Search MVP	PostgreSQL FTS + pg_trgm	Avoid premature search service; supports aliases/fuzzy matching.
Object storage	S3-compatible object storage	Signed upload/download, lifecycle deletion, vendor portability.
CV web	MediaPipe Tasks / Pose Landmarker	Pose landmarks and optional segmentation on supported web clients.
CV server	Python MediaPipe / model adapter	Fallback, batch reassessment and model experimentation.
Observability	OpenTelemetry + error/metrics/log backend	Vendor-neutral instrumentation.
CI/CD	GitHub Actions-class pipeline + IaC	Automated lint/test/build/migrate/deploy gates.

Version policy Pin production dependencies in lockfiles and container images. This TRD intentionally avoids hard-coding library version numbers; approved versions belong in the repository architecture decision records (ADRs) and dependency manifests.

3.2 Domain boundaries

Domain	Owns	Must not own
Identity	account, session, consent, device	workout or nutrition facts
Athlete profile	goals, mode, equipment, baseline metrics	food master data
Training	exercise catalogue, programs, sessions, sets, PRs	CV raw image processing
Nutrition	foods, aliases, recipes, meals, nutrient totals	training progression
Habits	habit definitions/schedules/completions	nutrition totals
CV assessment	capture metadata, derived features, model versions	medical diagnosis
Recommendations	decision records and explanations	source-of-truth workout history
Analytics	aggregates, snapshots, report payloads	authoritative transactional state

04 Frontend / PWA Architecture

4.1 Route map

Route group	Primary screens
/auth	register, sign-in, recovery, verification
/onboarding	mode, goals, metrics, experience, equipment, diet, CV consent/capture, plan preview
/today	daily action dashboard, readiness, water, habits, active workout
/training	program calendar, day view, workout logger, exercise detail, PR history
/nutrition	meal diary, food search, recipe builder, favorites, targets
/habits	month grid / Excel-style tracker, habit settings
/progress	body metrics, photos, CV assessments, weekly/monthly comparisons
/reports	weekly and monthly reports, exports
/settings	profile, units, privacy, consent, account/data controls
/admin	restricted operations console

4.2 State strategy

- Server state: fetched through typed API client; cache keys include athlete_id + relevant period/version.
- Workout-in-progress state: persist locally after every set; queue unsynced writes and reconcile when online.
- Form state: schema-validated client-side using the same constraints exposed by API/OpenAPI where practical.
- Global UI state: theme, selected date, sheet/dialog state only; do not mirror server entities into a custom global store.
- Charts: render aggregated payloads from report/dashboard endpoints rather than downloading all raw history.

4.3 Offline / weak-network behavior

Operation	Offline behavior	Conflict policy
Log set	Save local operation with UUID + client timestamp	Server accepts idempotently; latest valid edit wins with audit history.
Tick habit	Optimistic local toggle	Use completion date + habit ID uniqueness.
Log water	Queue append operation	Deduplicate by operation UUID.
Search global foods	Use recent/favorite cache only	Full search requires network.
Start planned workout	Cache today/next session payload	If plan changed remotely, warn at sync; never discard completed sets.

4.4 Accessibility and responsive requirements

- Keyboard-operable desktop habit grid and workout forms; visible focus states.
- Touch targets sized for between-set mobile use; one-handed primary controls.
- Charts require text summaries and accessible labels, not color-only meaning.
- All metrics show units and respect kg/lb + cm/in preferences.
- PWA installability and home-screen launch are P1; core web behavior is P0.

05 Backend & API Architecture

5.1 Application structure

Use a modular monolith: one deployable FastAPI application with domain packages. Each package exposes service functions and repository interfaces. Cross-domain writes happen through application services, not direct table access from unrelated modules.

```
app/
  api/              # routers, dependencies, request/response schemas
  domains/
    identity/
    athlete/
    training/
    nutrition/
    habits/
    cv_assessment/
    recommendations/
    analytics/
  infrastructure/   # DB, cache, object storage, mail/push adapters
  workers/          # async jobs / scheduled jobs
  common/           # IDs, time, errors, auth primitives
```

5.2 Transaction rules

- One user command should normally commit in one DB transaction.
- Derived analytics are not updated synchronously if doing so increases latency or lock contention; emit an outbox/event and aggregate asynchronously.
- Use database uniqueness constraints as the final defense for duplicate set logs, habit completions and import identifiers.
- No request should hold a database transaction open while waiting for an external API or model inference.
- All timestamps stored in UTC; athlete-local date is calculated using saved IANA timezone.

5.3 Error contract

Field	Example / rule
code	TRAINING_SESSION_ALREADY_COMPLETED
message	Human-readable safe message
details	Optional field-level validation details; no secrets/stack traces
request_id	Correlates logs/traces
retryable	Boolean for client retry UX

06 Identity, Accounts & Consent

ID	Requirement
IAM-001	Support email/password at P0; OAuth providers may be added behind a provider adapter.
IAM-002	Use secure HTTP-only cookies for web sessions or a short-lived access + rotating refresh-token design; never persist long-lived bearer tokens in localStorage.
IAM-003	Every API object access validates ownership/role; UUID identifiers do not replace authorization checks.
IAM-004	Separate consent records for Terms, Privacy, optional raw photo storage, analytics and future research/model-training use.
IAM-005	Consent withdrawal must stop future processing and queue deletion where applicable without deleting unrelated workout/nutrition history.
IAM-006	Account export and deletion jobs are auditable and idempotent.
IAM-007	Admin/support impersonation is disabled by default; if added, use explicit elevated session banners and immutable audit logs.
IAM-008	CV photo features should be age-gated in v1; legal review determines under-18 availability and guardian flows before release.

6.2 Roles

Role	Capabilities
athlete	Own profile, logs, reports, consent, exports/delete
support	Read limited operational metadata; no raw photos by default
content_admin	Manage exercise/food reference data
ops_admin	Jobs, source imports, feature flags, system health
coach (future)	Explicitly shared athletes only; scoped read/write permissions

07 Core Data Model

Identifiers are UUID/ULID-class opaque IDs. Mutable athlete measurements are append-only history rows wherever trend analysis matters. Soft delete is used only when business recovery requires it; privacy deletion must ultimately remove/anonymize data according to policy.

7.1 Entity catalogue

Entity	Purpose / key fields
users	account identity; status; timezone; created_at
athlete_profiles	mode, goal priorities, level, preferences, unit system
consents	consent_type, version, granted_at, revoked_at
body_metric_entries	weight, waist, optional measurements, source, measured_at
readiness_logs	sleep, soreness, energy, resting HR optional, recorded_date
equipment_items	canonical equipment definitions
athlete_equipment	availability + location
exercises	canonical exercise, movement pattern, muscles, modality
exercise_variants	equipment/skill/assistance relationships
programs	versioned generated or template plan
program_days	day index / scheduled date / intent
prescribed_exercises	sets, rep target, load rule, rest, notes
workout_sessions	actual session start/end/status/RPE
set_logs	set index, reps/load/time/distance/RIR/RPE/assistance
personal_records	record type/value/source session
food_sources	dataset/vendor/license/version/import status
foods	canonical per-100g nutrient item
food_aliases	regional name, spelling, transliteration, locale
serving_options	grams per serving + label
recipes	canonical or user recipe
recipe_ingredients	food/recipe relation + quantity
meal_logs	date, meal_type, notes
meal_items	food/recipe, quantity, nutrient snapshot
nutrition_targets	date range, calorie/macro behavior targets
water_logs	amount_ml + timestamp
habits	name, schedule, target, active period
habit_completions	habit_id + local_date + value/status
streak_snapshots	computed current/best streak + through_date
progress_media	object key, capture type, retention flags
cv_assessments	model version, quality, feature vector, confidence
recommendation_decisions	inputs snapshot, rule/model version, action, explanation
weekly_reports	period + materialized report payload
notifications	channel, template, status, scheduled_at
audit_logs	actor, action, entity, request_id, metadata

7.2 Important constraints / indexes

Table	Constraint / index
set_logs	UNIQUE(workout_session_id, prescribed_exercise_id, set_index) unless explicit extra-set flag.
habit_completions	UNIQUE(habit_id, athlete_id, local_date).
foods	Index normalized_name; trigram index; source_external_id unique per source.
food_aliases	Trigram/unaccent index on normalized alias; locale/cuisine filters.
body_metric_entries	Index (athlete_id, measured_at DESC).
workout_sessions	Index (athlete_id, started_at DESC) and status.
meal_logs	Index (athlete_id, local_date, meal_type).
recommendation_decisions	Index athlete_id + effective_at; immutable after applied except status fields.
cv_assessments	Feature schema version + model version required; never store unlabeled vectors.

08 Nutrition Data Platform

8.1 Source strategy

Use a normalized internal food schema so no external source controls the application model. USDA FoodData Central can provide broad nutrient data through its API/downloads. Indian food coverage is created from legally usable Indian food-composition/reference data plus a curated recipe layer and aliases. Packaged-food sources can be added through a source adapter.

Licensing gate Before importing any third-party Indian food table or packaged-food dataset, engineering must record source terms, attribution requirements, redistribution restrictions and update policy in food_sources. Do not assume that "publicly viewable" means freely redistributable.

8.2 Canonical food schema

Field	Type / example	Notes
canonical_name	text: Bhindi sabzi	Primary display name
normalized_name	text: bhindi sabzi	Search key
locale_names	relation / aliases	Hindi/English/transliterations and regional variants
food_type	ingredient | dish | packaged	
cuisine	north_indian, south_indian, global...	multi-tag allowed
diet_type	veg / egg / nonveg / vegan compatible	derived carefully
state	raw / cooked / prepared	distinct records when nutrients differ
basis_g	100	nutrient normalization basis
energy_kcal	numeric	per basis
protein_g	numeric	per basis
carb_g	numeric	per basis
fat_g	numeric	per basis
fiber_g	numeric optional	
micronutrients	normalized nutrient rows or JSON snapshot	prefer nutrient dimension table if extensive
servings	relation	e.g. 1 katori = estimated grams
source_id/version	foreign key	traceability
data_quality	verified / curated / user-generated	controls ranking

8.3 Indian search dataset requirements

- Include canonical dishes plus aliases: "aloo matar", "alu matar", "आलू मटर" if Devanagari support is enabled; "bhindi", "okra sabzi", "bhindi masala".
- Represent recipe variability explicitly. A dish record is a reference estimate; users can save a household recipe for better repeatability.
- Create common serving vocabulary: katori, bowl, cup, roti, chapati, piece, dosa, idli, ladle, glass, tablespoon, teaspoon.
- Tag common preparation methods: dry sabzi, gravy, fried, roasted, steamed, boiled, tandoor, curry.
- Ranking should prefer user recents/favorites, locale/cuisine match and verified data before fuzzy text similarity.

8.4 ETL pipeline

1. Fetch/download source into immutable staging snapshot.
2. Validate source/version/license metadata.
3. Parse nutrients and units; reject impossible ranges.
4. Normalize names, units and per-100g basis.
5. Map nutrient identifiers to canonical nutrient dimension.
6. Generate aliases/transliterations only from reviewed rules; preserve source name.

7. Deduplicate using source IDs + fuzzy review queue, not automatic destructive merge.
8. Publish approved snapshot; mark superseded records without deleting historical meal nutrient snapshots.

8.5 Meal logging calculation

Each meal item stores a nutrient snapshot at log time. Historical daily totals therefore do not change if the master food record is later corrected. Calculation: nutrient_amount = nutrient_per_100g × consumed_grams / 100. Recipe nutrition is the sum of ingredient snapshots adjusted for recipe yield.

09 Workout & Progression Engine

9.1 Exercise schema

Attribute	Examples
modality	weighted_reps, bodyweight_reps, assisted_reps, weighted_bodyweight, isometric_hold, distance_time, interval
movement_pattern	horizontal_push, vertical_pull, squat, hinge, carry, sprint, jump, core, skill
primary/secondary muscles	normalized muscle IDs
equipment	barbell, dumbbell, cable, rings, pull-up bar, machine, bodyweight
skill tier	foundation → intermediate → advanced
contraindication tags	non-medical caution metadata / movement limitations
tracking fields	load, reps, seconds, distance, assistance, RIR, RPE, tempo
substitution group	equivalent intent group for unavailable equipment

9.2 Program generation inputs

- athlete mode and weighted goal priorities
- system level + training age
- available days and session duration
- equipment inventory and training location
- exercise preferences/avoidances
- recent weekly sets by muscle/pattern
- performance trend and plateau markers
- readiness/recovery trend
- calisthenics skill prerequisites
- injury/pain exclusions supplied by the user (no diagnosis)

9.3 Deterministic progression rules - v1

Case	Example rule
Rep-range / double progression	When all work sets reach top of target range at acceptable RIR for two exposures, increase load by configured increment and reset reps toward lower range.
Bodyweight calisthenics	Progress reps → tempo/ROM → reduced assistance → harder leverage/variation; preserve prerequisite graph.
Isometric skill	Increase total quality hold time; progress leverage only after minimum hold quality threshold.
Athletic intervals	Progress volume or intensity, not both simultaneously, within template constraints.
Poor recovery	Do not auto-add volume. Hold or reduce prescribed effort when repeated low readiness + elevated session RPE is observed.
Missed sessions	Reschedule intelligently; do not compress all missed volume into the next day.
Plateau	Require multiple exposures and adequate adherence before changing exercise/volume.
Deload candidate	Trigger review when fatigue indicators + performance regression persist; deload rules are configurable by mode/level.

9.4 Set logging model

The logger supports different set types without forcing fake zeros. A weighted set may use load+reps; a plank uses seconds; a sprint uses distance+time; assisted pull-ups use reps+assistance. The API validates only fields required by that exercise tracking schema.

Metric	Derived use
volume_load	Σ load × reps; useful but not universal
hard_sets	sets meeting configured effort criteria
rep PR	best reps at same load/variation
load PR	heaviest successful load
e1RM optional	trend estimate from eligible weighted exercises only
skill volume	hold seconds / quality reps / assistance level
adherence	completed prescribed work ÷ planned work
session RPE	global fatigue/perceived difficulty signal

10 Personalization / Recommendation Engine

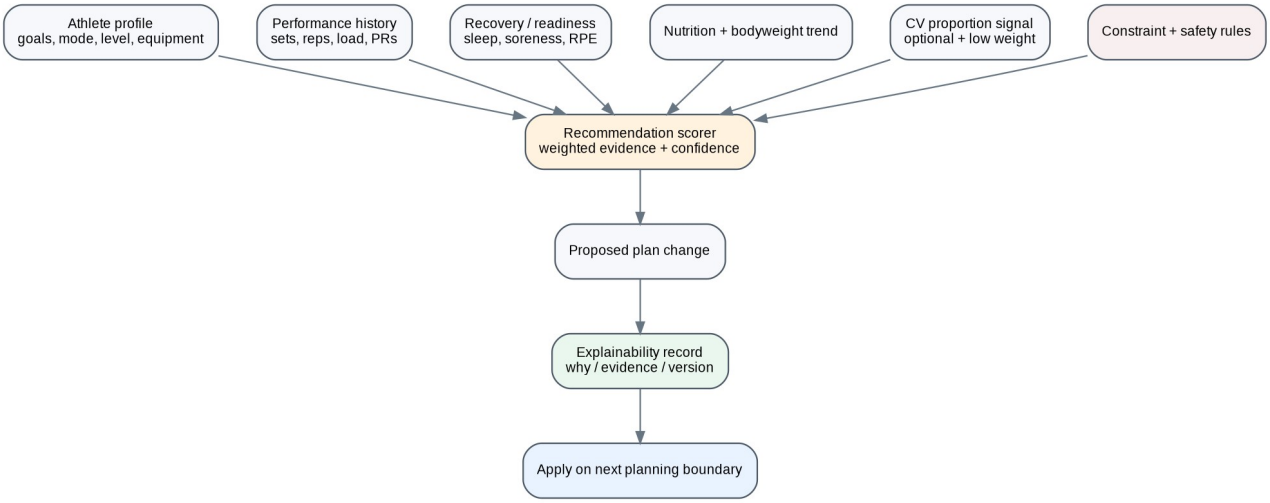

Figure 2. Recommendation engine uses multiple evidence sources; CV is optional and intentionally low-weight.

10.1 Architecture

V1 should be a rules + scoring system, not an unconstrained generative model. A recommendation candidate is created, validated against constraints, assigned confidence, explained, and only then applied at an appropriate planning boundary.

Input family	Typical weight / role
Performance history	Highest - objective response to prescribed training
Adherence / consistency	High - prevents escalation when execution is insufficient
Recovery / readiness	High - controls fatigue and session demand
Bodyweight / measurements trend	Medium - goal-response signal
Nutrition adherence	Medium - context for bodyweight/performance trends
CV proportion/change features	Low/secondary - visual context only; never sole driver
User preferences	Hard constraints + ranking modifiers
Safety/availability constraints	Hard constraints; cannot be overridden by score

10.2 Decision record

Field	Purpose
decision_type	e.g. increase_load, reduce_volume, swap_exercise, hold_plan
effective_at	when change may take effect
evidence_snapshot	immutable metrics/window used
rule_set_version	reproducibility
model_version	nullable for pure rules
confidence	0-1 or categorical calibrated band
explanation	user-facing reason
safety_checks	list of validators passed/failed
status	proposed, applied, rejected, user_overridden
override_reason	optional athlete/coach feedback

10.3 LLM usage - optional P2/P3

- Allowed: explain a deterministic decision in plain language, summarize a weekly report, answer questions using the athlete's approved data scope.

- Not allowed: silently invent exercises, nutrition facts, medical advice or change persistent plans without passing structured validators.
- Use retrieval of canonical exercise/food/plan facts; output must be converted into typed actions and validated before storage.
- Keep prompts/version IDs and redacted evaluation samples for quality testing; never use private athlete data for model training without separate consent/legal basis.

11 Computer Vision Assessment

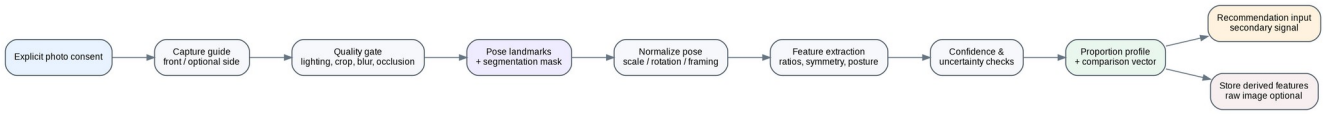

Figure 3. Privacy-conscious CV pipeline for pose/proportion analysis.

11.1 Technical goal

Convert a standardized upper-body progress image into reproducible pose/proportion features that can support progress comparison and weak-point context. The system does not infer a medically accurate body-fat percentage, diagnosis or immutable “body type.”

11.2 Recommended v1 model

Use MediaPipe Pose Landmarker in the browser where supported. Enable segmentation output when required for silhouette-based width measurements. Keep a Python adapter with the same feature schema for server fallback/batch processing. All downstream code consumes an internal AssessmentFeatureSchema, not MediaPipe-specific objects.

11.3 Capture protocol

- Front pose required; optional side pose for posture/progress. Camera approximately chest height and level.
- Consistent distance, neutral stance, arms slightly separated from torso when capture guide requests it.
- Normal sportswear/fitted clothing. The app must not request nude imagery.
- Quality gate checks blur, crop, severe occlusion, landmark confidence and major body rotation.
- Face region should not be used for identification; optional client-side face blur can be added before storage.

11.4 Derived feature schema

Feature group	Examples	Caveat
Pose geometry	shoulder/hip line angle, torso inclination, landmark symmetry	2D camera perspective sensitive
Normalized proportions	shoulder width : torso/waist silhouette width, torso length ratios	not circumference
Symmetry	left/right shoulder level, arm/torso spatial symmetry	pose quality dependent
Silhouette trend	normalized widths at standardized torso y-levels	clothing affects outline
Progress comparison	same-feature delta vs prior standardized captures	only compare if quality/confidence compatible
Quality/confidence	landmark score, segmentation quality, pose deviation	must accompany every feature vector

11.5 Feature extraction algorithm

9. Decode image locally and downscale to bounded processing resolution without changing aspect ratio.
10. Run pose model; reject if required torso landmarks are below configured confidence.
11. If segmentation is enabled, obtain person mask and remove small disconnected components.
12. Normalize coordinates using shoulder midpoint/hip midpoint axis; compensate for mild roll.
13. Compute landmark distances as ratios to torso/shoulder reference lengths rather than pixels.
14. Sample silhouette widths only in anatomically bounded zones derived from landmarks.
15. Calculate capture-quality metrics and confidence per feature family.
16. Emit versioned derived feature JSON. Upload raw image only if separate storage consent is active.

17. Compare with compatible previous assessment; suppress claims if capture conditions/confidence differ too much.

11.6 Model evaluation

- Create consented evaluation set across different body sizes, skin tones, lighting, camera devices and normal training clothing.
- Measure landmark availability, repeatability across duplicate captures, silhouette stability and false change rate.
- Product KPI is repeatability/usefulness, not “body-fat accuracy.”
- If feature confidence is poor, return “capture again” rather than forcing a classification.

12 Habit & Streak Engine

12.1 Habit model

Field	Examples
frequency	daily, selected_weekdays, N_per_week
measurement	boolean, count, minutes, ml
target	1 completion, 8000 steps, 3000 ml
schedule_start/end	active period
grace policy	none by default; optional streak-freeze is a product feature, not hidden logic
timezone	athlete-local calendar day
source	manual, derived from water/workout, future wearable

12.2 Excel-style grid behavior

- Rows = habits; columns = local calendar days; current month is default.
- Boolean habit uses checkbox; measured habit displays progress and allows quick entry.
- Keyboard navigation on desktop and sticky first column/date header where screen size allows.
- Derived habits (e.g. “Workout completed”) are read-only from the habit grid unless product explicitly allows manual override.
- Grid API should fetch one month at a time using a compact matrix payload.

12.3 Streak computation

Streaks are computed from the habit schedule, not consecutive UTC dates. For a Monday/Wednesday/Friday habit, missing Tuesday is irrelevant. Recompute asynchronously after schedule edits and store snapshots for fast UI; authoritative completion rows remain the source of truth.

13 Reporting & Analytics

13.1 Dashboard aggregates

Area	Weekly metrics	Monthly comparison
Training	sessions completed, sets, volume, rep/load PRs, adherence, average session RPE	current month vs prior month; muscle/pattern volume trend
Body	weight trend, measurement deltas	monthly averages and rate of change
Nutrition	days logged, calories/macros vs target, meal consistency	adherence and bodyweight context
Hydration	average ml/day, target days achieved	consistency
Habits	completion %, current/best streak	habit-by-habit comparison
Recovery	sleep/readiness averages	trend and relation to performance
CV optional	standardized feature delta + capture confidence	comparable scans only

13.2 Analytics architecture

- Transaction tables remain normalized; reporting uses SQL views/materialized aggregates or precomputed report payloads.
- Weekly report job runs after athlete-local week boundary or on demand. Month comparisons use calendar months in athlete timezone.
- Every chart endpoint returns the data range, units and missing-data indicators.
- Do not compute a single “fitness score” as the only summary. Keep training, adherence, recovery, nutrition and body progress separable.
- Recommendation explanations can reference report metrics by stable IDs so UI can deep-link to evidence.

14 Search, Cache & Background Processing

14.1 Food search

MVP search uses normalized text + PostgreSQL full-text/trigram matching. Query pipeline: normalize Unicode → lower-case → unaccent/transliteration lookup → exact alias → prefix → trigram similarity → ranking by user history/source quality/locale.

Ranking signal	Priority
exact canonical/alias match	very high
recently logged by athlete	high
favorite / saved recipe	high
verified curated source	high
locale/cuisine preference	medium
prefix match	medium
fuzzy similarity	fallback
user-generated global item	lower unless owner

14.2 Cache policy

Cache key family	TTL / invalidation
food_search:<normalized_query>:<locale>	short TTL; source publish version included
exercise_catalog:<filters>	minutes-hours; invalidate on content publish
dashboard:<athlete>:<date>	short TTL; invalidate on relevant log write
today_plan:<athlete>:<date>	invalidate on plan adaptation or reschedule
rate_limit:*	sliding/fixed window counters
distributed_lock:*	seconds-minutes; jobs only

14.3 Background jobs

- food dataset import/normalization
- weekly/monthly report materialization
- streak recomputation after schedule edits
- notification scheduling/fan-out
- export/account deletion
- progress media retention deletion
- recommendation refresh at planning boundary
- analytics event compaction
- model reprocessing only with appropriate consent and version controls

15 Media & File Storage

Requirement	Implementation
MEDIA-001	Use private buckets by default. Access through short-lived signed URLs after authorization.
MEDIA-002	Object key must not expose email/phone/name; use opaque athlete and media IDs.
MEDIA-003	Store MIME type, byte size, checksum, capture purpose, consent ID, created_at and retention state.
MEDIA-004	Strip EXIF/location metadata on client or trusted ingestion path before long-term storage.
MEDIA-005	Raw photo storage is optional. Derived CV features remain independently deletable/recomputable according to policy.
MEDIA-006	Lifecycle worker deletes revoked/expired objects and verifies deletion status.
MEDIA-007	Exports are encrypted at rest, short-lived and removed automatically after configured period.

16 API Contract

REST/JSON under /api/v1 for v1. OpenAPI is generated from FastAPI schemas. Dates are ISO-8601; timestamps use UTC offsets; weights and lengths are transmitted with explicit units or canonical SI storage semantics.

Domain	Endpoint	Purpose
Profile	GET /me	current athlete/account summary
Profile	PATCH /me/profile	update goals/preferences
Metrics	POST /body-metrics	append measurement
Metrics	GET /body-metrics?from=&to=	history
Readiness	PUT /readiness/{date}	daily check-in
Equipment	PUT /me/equipment	replace availability set
Training	GET /programs/active	active program
Training	GET /training/days/{date}	planned day
Training	POST /workouts	start session
Training	POST /workouts/{id}/sets	log set idempotently
Training	PATCH /workouts/{id}/sets/{set_id}	edit set
Training	POST /workouts/{id}/complete	complete session
Training	GET /exercises/search?q=	exercise search
Training	GET /personal-records	PR list
Nutrition	GET /foods/search?q=	food/alias search
Nutrition	GET /foods/{id}	food detail
Nutrition	POST /recipes	create user recipe
Nutrition	PUT /meals/{date}/{type}	create/update meal shell
Nutrition	POST /meals/{id}/items	add food/recipe
Nutrition	PATCH /meal-items/{id}	edit quantity
Nutrition	DELETE /meal-items/{id}	remove item
Nutrition	GET /nutrition/days/{date}	daily totals
Water	POST /water	append water entry
Habits	GET /habits	list definitions
Habits	POST /habits	create habit
Habits	PUT /habits/{id}	edit schedule
Habits	PUT /habits/{id}/days/{date}	set completion
Habits	GET /habit-grid?month=YYYY-MM	matrix payload
CV	POST /cv/assessments	submit derived features or upload intent
CV	GET /cv/assessments	history
CV	DELETE /progress-media/{id}	delete stored media
Reports	GET /dashboard?period=week	dashboard aggregate
Reports	POST /reports/weekly/{date}/generate	on-demand generation
Reports	GET /reports/weekly/{date}	report payload
Recommendations	GET /recommendations	current decisions/explanations
Recommendations	POST /recommendations/{id}/override	user override feedback
Account	GET /privacy/consents	consent state
Account	PUT /privacy/consents/{type}	grant/revoke
Account	POST /account/export	queue export
Account	DELETE /account	queue deletion
Admin	POST /admin/food-imports	start source import
Admin	GET /admin/jobs/{id}	job status
Admin	PUT /admin/exercises/{id}	edit/publish exercise
Admin	GET /admin/data-quality	quality queues

16.2 API mechanics

- Mutation endpoints accept Idempotency-Key where duplicate network retries are likely.
- Pagination uses cursor-based pagination for long histories/search results.
- ETags/If-None-Match may be used for slowly changing reference catalogues.
- 429 responses include retry guidance; rate limits differ for auth, search, CV upload and normal writes.
- API never exposes raw database exception text.

17 Security & Privacy

Security verification should be mapped to OWASP ASVS and API Security guidance. Athlete data access is object-scoped: every request that references an athlete-owned object must enforce ownership/role checks regardless of identifier entropy.

Control family	Requirement
Transport	TLS only; HSTS in production; secure cookies; no mixed content.
Auth	Brute-force controls, credential reset protection, optional MFA/passkeys later.
Authorization	Object-level authorization on every athlete resource; admin scope separation.
Input	Schema validation, upload MIME/size checks, image decode verification, query parameter bounds.
Secrets	Secret manager/env injection; never committed; rotation runbook.
Database	Least-privilege app roles; migration role separate; consider RLS as defense-in-depth if deployment model supports it.
API abuse	Rate limits, request-size limits, expensive-search/CV quotas, pagination caps.
Photos	Private bucket, signed URLs, EXIF stripping, explicit retention/consent, no public object ACLs.
Logging	PII minimization; never log auth tokens, raw photos, passwords or full sensitive request bodies.
Dependencies	Automated vulnerability scanning + lockfile review; patched base images.
Admin	MFA, restricted network/session controls, audit logs, least privilege.
Privacy	Data export, deletion, purpose-specific consent, retention schedule; legal review for applicable jurisdictions before production.

17.2 Threat scenarios to test

- Changing workout_session_id to read/edit another athlete’s sets (BOLA).
- Enumerating progress_media IDs or signed-URL leakage.
- Credential stuffing and password reset abuse.
- Unbounded food-search requests causing resource exhaustion.
- Malicious image decompression / oversized uploads.
- CSRF on session-authenticated mutations.
- Admin content injection/XSS in exercise or food descriptions.
- Replay of offline set-log requests creating duplicates.
- Deleted account data lingering in caches, exports or object storage.
- Prompt injection in future AI coach content attempting to bypass structured action validation.

18 Performance, Scalability & Reliability

NFR	Target / design requirement
NFR-001 API latency	Common authenticated reads p95 <= 300 ms excluding cold starts/external calls; writes p95 <= 700 ms under normal load.
NFR-002 Food search	Warm search response target <= 250 ms for common queries at MVP data scale.
NFR-003 Workout durability	Set entry is persisted locally immediately and server-acknowledged when online; no completed set lost due to navigation.
NFR-004 Availability	Design for 99.9% monthly API availability after production hardening; MVP may use lower-cost single-region managed services.
NFR-005 Backups	Automated DB backups + point-in-time capability where provider supports it; restore drills scheduled.
NFR-006 RPO/RTO	Production target: RPO <= 15 min, RTO <= 2 hr for primary transactional store; validate against provider capabilities.
NFR-007 Scale	Stateless API/web tiers horizontal; workers scale by queue depth; object storage externalized.
NFR-008 Payloads	Dashboard/report endpoints return aggregates; avoid multi-megabyte raw history payloads.
NFR-009 CV	On-device inference must show progress/cancel state and fail gracefully to manual metrics if unsupported.
NFR-010 Data consistency	Strong consistency for user writes; eventual consistency acceptable for analytics, streak snapshot and report refresh.

18.2 Scale transition triggers

- Split nutrition search/indexing when database search load materially impacts transactional workload.
- Split CV server inference when GPU/CPU scaling profile diverges from API.
- Move analytics to separate warehouse when report queries/retention exceed operational DB comfort.
- Introduce read replicas after query profiling shows clear read pressure; not preemptively.
- Adopt event bus beyond Redis queue when cross-service event durability/replay becomes a business requirement.

19 Observability & Operations

Signal	Required dimensions / examples
Logs	request_id, actor/athlete pseudonymous ID, route, status, latency, domain event; no sensitive payloads
Metrics	request count/latency/error, DB pool, queue depth, worker failures, search latency, import quality, CV failures
Traces	web/API/DB/worker spans for slow requests and report jobs
Product events	onboarding complete, workout started/completed, set logged, food search zero-result, habit tick, report viewed
Data-quality metrics	duplicate foods, missing nutrients, alias hit rate, recipe source coverage
Model metrics	capture rejection rate, landmark confidence distribution, feature repeatability, recommendation override rate

19.2 Alerts

- 5xx/error-rate spike
- DB connection/pool saturation
- queue backlog age above threshold
- food import job repeated failure
- object deletion job failure
- auth anomaly / rate-limit surge
- CV failure rate regression after model release
- report generation latency/error regression

20 Testing & Quality Strategy

Layer	Coverage
Unit	progression rules, streak schedule math, nutrition calculations, normalization, authorization helpers
Property-based	unit conversion, date/timezone boundaries, nutrient scaling, idempotency
API integration	real PostgreSQL/Redis test services; auth ownership and transaction rollback
Contract	OpenAPI schema snapshots; generated frontend client compatibility
Frontend	component + route tests; mobile workout logger; offline queue reconciliation
E2E	onboarding → plan → workout → meal → habit → report happy path and failure paths
Security	SAST/dependency scan, authz tests, upload abuse tests, OWASP-oriented DAST in staging
CV	golden capture set, quality rejection, repeatability, unsupported-browser behavior
Data ETL	source fixtures, nutrient/unit validation, dedupe review, source version rollback
Performance	food search, set logging bursts, dashboard/report load, queue throughput
Accessibility	automated checks + keyboard/screen-reader manual checks on core routes

20.2 Critical automated test cases

- Two retries with same Idempotency-Key create one set log.
- Athlete A cannot fetch/update/delete Athlete B object IDs.
- Changing timezone near midnight does not duplicate/lose habit day semantics.
- Updating a master food nutrient record does not alter historical meal snapshot totals.
- Completed workout session cannot be accidentally regenerated into a blank session.
- CV low-confidence capture creates no proportion recommendation.
- Revoking raw-photo consent queues media deletion while retaining permitted derived/non-photo history.
- Recommendation engine records evidence and reason for every applied change.

21 CI/CD, Environments & Deployment

21.1 Environments

Environment	Purpose	Data policy
local	developer services via containers	synthetic seed data only
preview	per-PR frontend/API preview where economical	synthetic data
staging	production-like integrations, migrations, security/perf tests	synthetic or explicitly approved test accounts
production	real athlete data	strict access, backups, audit, monitoring

21.2 Pipeline gates

18. format + lint + type-check
19. unit tests
20. build frontend/backend images
21. dependency/SAST scan
22. integration tests with disposable DB
23. OpenAPI compatibility check
24. migration dry-run / expand-contract validation
25. deploy staging
26. smoke + E2E + security checks
27. manual/automated production approval depending release class
28. post-deploy health checks and rollback readiness

21.3 Deployment profiles

Profile	Suggested shape
Lean MVP	Frontend on managed Next.js platform; FastAPI container on managed app platform; managed PostgreSQL + Redis; S3-compatible storage.
Scale production	CDN/edge frontend; containerized API/workers behind load balancer; managed HA PostgreSQL/Redis; private object storage; centralized telemetry; IaC.
Enterprise future	Private networking, dedicated keys, regional data controls, SSO/SCIM if B2B coach/gym offering requires it.

22 MLOps & Data Science Lifecycle

Artifact	Version requirement
CV model	model package/version + runtime + feature-schema version
CV quality thresholds	configuration version
Recommendation rule set	semantic version + effective date
Statistical/ML model	training data version, features, metrics, artifact hash
Food normalization rules	pipeline version + source snapshot IDs
Experiment	hypothesis, target metric, cohort rules, start/end, analysis

22.2 Release process for model/rule changes

- Offline evaluation on versioned validation fixtures.
- Shadow mode in staging/production where candidate decisions are logged but not applied.
- Compare override rate, safety-validator failures, capture rejection and outcome proxies.
- Canary by feature flag; retain immediate rollback to previous model/rule set.
- Never recompute historical recommendation explanations without preserving the version originally used.

22.3 Data governance

- Operational athlete data is not automatically a training dataset.
- If future model training uses user data, require documented purpose, minimization, consent/legal basis and deletion propagation strategy.
- De-identification should remove direct identifiers and minimize quasi-identifiers; raw photos are a separate high-sensitivity class.
- Evaluation sets must record capture conditions and model limitations, not demographic profiling.

23 Admin / Support Engineering

Admin module	Functions
Exercise catalogue	create/edit/version exercises, substitutions, tracking schema, publish status
Food data	source imports, alias review, duplicate queue, nutrient anomaly queue, publish snapshot
Recipe curation	approve canonical Indian/global recipes, serving estimates, source attribution
User support	account status and request IDs; restricted access to personal data
Jobs	view/retry/cancel eligible background jobs; poison queue visibility
Feature flags	rollout CV, recommendations, new nutrition source, experimental UI
Audit	search immutable admin actions and consent/data-deletion events
Operations	system health, import age, queue backlog, storage deletion verification

24 Failure Modes & Recovery

Failure	Expected behavior
Food source unavailable	Serve last published internal snapshot; import job retries with backoff; user logging unaffected.
Redis unavailable	Degrade caches/rate limits according to fail policy; transactional API should not lose data; async jobs may pause.
CV unsupported/browser failure	Offer manual progress metrics and retry guidance; no blocking of onboarding.
Object storage failure	Derived-only CV may continue; raw media upload fails safely without creating dangling “stored” record.
Recommendation job failure	Keep current valid plan; surface no speculative changes; retry async.
Report job failure	Dashboard raw aggregates remain available; report status shows retryable error.
Duplicate offline writes	Idempotency/unique keys collapse duplicates.
DB primary failure	Managed failover/restore according to provider; clients receive safe retryable errors.
Bad food import	Publish is atomic/versioned; rollback to previous catalogue snapshot.
Bad recommendation release	Feature flag/roll back rule/model version; existing applied decision remains auditable.

25 Implementation Plan & Team Ownership

Phases are sequencing guidance, not a commitment to calendar dates. Build vertical slices so the product is usable before advanced AI is introduced.

Phase	Deliverables	Primary owners
0 - Foundation	repo, CI, environments, auth, DB migrations, observability skeleton	platform + full-stack
1 - Core athlete loop	onboarding/profile, exercise catalogue, program/day, workout/set logger, basic dashboard	frontend + backend
2 - Nutrition + habits	food schema/search, curated Indian seed data, meals, recipes, water, habit grid/streaks	backend/data + frontend
3 - Reporting	weekly/monthly aggregates, charts, exports, recommendation evidence framework	backend/data + frontend
4 - CV assist	capture UI, MediaPipe inference, quality gates, versioned feature storage, progress comparison	CV/ML + frontend
5 - Adaptive engine	rules/scoring, recovery-aware progression, explanation records, override feedback	training domain + data/ML
6 - Production hardening	security tests, performance, backups/restore, deletion/export, admin/ops	platform/security + all
7 - Optional intelligence	AI coach explanation, wearables, exercise-form modules, coach portal	future squads

25.2 Suggested ownership boundaries

- Frontend engineer: PWA shell, workout logger, habits grid, charts, CV capture UX.
- Backend engineer: domain services, API, PostgreSQL, authz, background jobs.
- Data/ML engineer: nutrition ETL/search quality, CV feature pipeline, recommendation evaluation.
- Product/fitness domain reviewer: exercise taxonomy, templates, progression safety constraints and content QA.
- Platform/security: CI/CD, infrastructure, secrets, backups, security verification and observability.

26 Repository Structure & Engineering Conventions

```
athleteos/
  apps/
    web/                  # Next.js PWA
    api/                  # FastAPI application
    worker/               # background worker entrypoint
  packages/
    api-client/           # generated/typed client
    domain-types/         # shared enums / JSON schemas where appropriate
    ui/                   # design-system components
  data/
    fixtures/             # synthetic food/exercise/test fixtures
    schemas/              # import schemas, feature schemas
  ml/
    cv/                   # model adapters + evaluation
    recommendations/      # rules/models/evaluation
  infra/
    containers/
    terraform-or-equivalent/
  docs/
    adr/                  # architecture decision records
    api/
    runbooks/
```

26.2 Conventions

- Use ADRs for significant choices: auth provider, queue, storage, search, CV runtime, deployment platform.
- Domain service methods use explicit commands/results; avoid “god service” utilities.
- All DB migrations are forward-reviewed and production-safe; destructive changes use expand/backfill/contract.
- Feature flags for new CV/recommendation behavior and risky migrations.
- No business rule exists only in frontend code; server validates authoritative rules.
- Synthetic fixtures never contain copied real athlete data.

27 Technical Acceptance / Definition of Done

Area	Release gate
Authentication	verified registration, reset, logout, session expiry and cross-user access tests pass
Onboarding	required fields persist; plan can be generated without CV; consent states versioned
Workout	day opens quickly; each set can be logged/edited offline-ish; sync idempotent; session completion durable
Nutrition	Indian dish aliases searchable; serving/grams conversion correct; historical nutrient snapshots stable
Habits	month grid correct across timezone/month boundaries; streak respects schedule
Reports	week/month comparisons reconcile to source logs within defined eventual-consistency window
CV	quality gate + derived features + confidence; raw image optional; no medical/body-fat claims
Recommendations	every applied plan change has evidence, version, explanation and safety validation
Security	object-level authorization tests, upload controls, secret scan, dependency scan and staging DAST complete
Privacy	consent revoke, media deletion, account export/delete flows verified
Ops	dashboards/alerts, backup/restore runbook and rollback documented
Performance	agreed p95 targets tested on representative seed dataset

28 PRD Traceability Matrix

PRD requirement family	TRD sections	Technical realization
PRD Auth & account	06, 16, 17	IAM + account endpoints + authz tests
PRD Athlete baseline	07, 16	profiles/body metrics/readiness schema
PRD CV assessment	11, 15, 17, 22	capture pipeline, media consent, model versioning
PRD Nutrition tracker	08, 14, 16	food ETL/search/meal APIs
PRD Training module	09, 10, 16	program/session/set model + progression engine
PRD Streaks	12	schedule-aware completions + snapshots
PRD Habit spreadsheet	04, 12	responsive grid + monthly matrix API
PRD Weekly/monthly dashboard	13	aggregates/report jobs/chart contracts
PRD UI/UX	04	mobile-first route/state/offline/accessibility requirements
PRD Security/privacy	06, 15, 17	consent, access control, media handling, deletion/export
PRD Admin/operations	19, 23	telemetry, jobs, catalogue/data-quality tools
PRD Release plan	21, 25, 27	CI/CD, sequencing and technical DoD

29 Reference Sources

Primary technical references consulted for this TRD. Product-specific choices remain architecture recommendations and should be recorded in repository ADRs before implementation.

Source	URL	Use
Google AI Edge - MediaPipe Pose Landmarker	https://ai.google.dev/edge/mediapipe/solutions/vision/pose_landmarker	Pose landmarks, web/Python deployment and optional segmentation-related task capabilities.
Google AI Edge - Pose Landmarker for Web	https://ai.google.dev/edge/mediapipe/solutions/vision/pose_landmarker/web_js	Browser implementation reference.
USDA FoodData Central API Guide	https://fdc.nal.usda.gov/api-guide	Programmatic nutrient data integration.
USDA FoodData Central Data Documentation	https://fdc.nal.usda.gov/data-documentation	Food data types and source interpretation.
Next.js App Router Documentation	https://nextjs.org/docs/app	Web application architecture reference.
FastAPI Documentation	https://fastapi.tiangolo.com/	Typed Python API and OpenAPI framework.
FastAPI Background Tasks	https://fastapi.tiangolo.com/tutorial/background-tasks/	Background work guidance; heavy workloads should be moved to worker infrastructure.
OWASP ASVS	https://owasp.org/www-project-application-security-verification-standard/	Security requirements and verification framework.
OWASP API Security	https://owasp.org/www-project-api-security/	API-specific authorization and abuse risk guidance.
PostgreSQL Row Security	https://www.postgresql.org/docs/current/ddl-rowsecurity.html	Optional database defense-in-depth reference.
Redis Documentation	https://redis.io/docs/latest/	Cache/queue infrastructure reference.

Final engineering note The fastest path to a reliable AthleteOS is to perfect the daily logging loop first. CV and adaptive intelligence should consume clean longitudinal data; they should not compensate for weak core data modeling or unreliable workout/nutrition logging.